

Constant Propagation

Definizione del problema:

La Constant Propagation è un'analisi che ha l'obiettivo di determinare, in un determinato punto del programma, quali variabili e il loro valore è essere considerato costante durante l'esecuzione. Inoltre, permette di ricavare il valore costante di espressioni binarie in cui uno o entrambi gli operandi sono variabili per le quali è già conosciuto il valore costante.

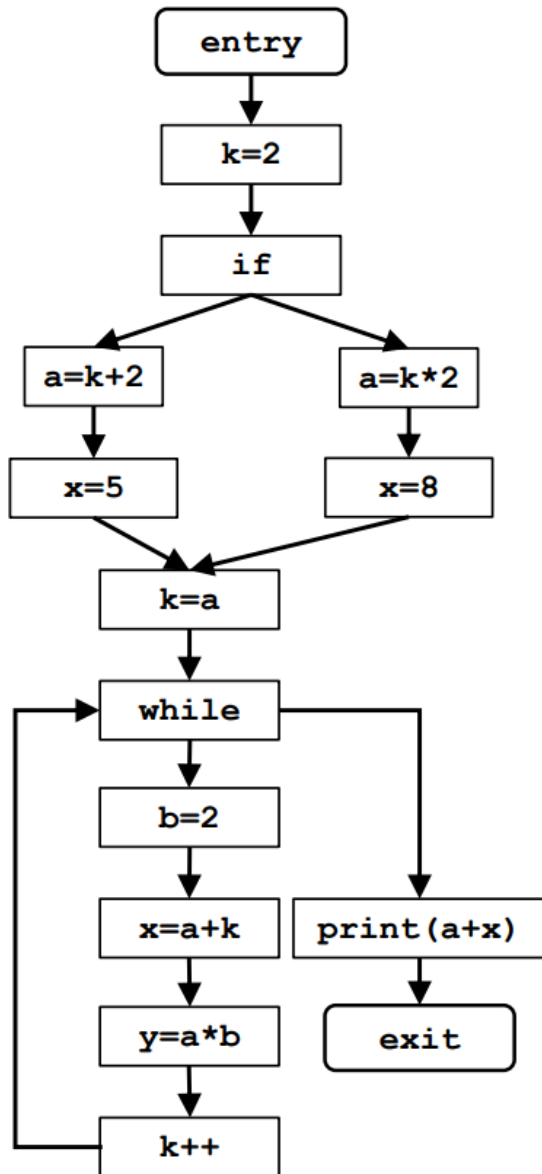
Framework:

	Dataflow problem: Constant Propagation
Domain	Sets of variables -> <variabile, valore>
Direction	Forward: $out[b] = f_b(in[b])$ $in[b] = \Delta out[pred(B)]$
Transfer function	$f_b(x) = gen_b \cup (in[b] - kill_b)$
Meet Operator Δ	$\cap$
Boundary Condition	$out[ENTRY] = \{\}$
Initial interior Points	$out[b] = \{\}$

Il dominio è rappresentato come <variabile, valore>, Questa rappresentazione permette di associare a ogni variabile un eventuale valore costante noto in un certo punto del programma. (ad es: <x,5> , <y,10>) il dominio deve essere abbastanza espressivo da distinguere: variabili con valore costante noto, variabili con valore non ancora determinato, variabili che possono assumere valori differenti lungo cammini diversi. La direzione scelta è Forward perchè la propagazione delle costanti segue il flusso naturale del programma: un'assegnazione fatta in un'istruzione influenza le istruzioni successive. Il meet operator è l'intersezione perché una coppia del dominio può essere considerata costante solo se assume lo stesso valore noto su tutti i cammini possibili.

La Boundary Condition è rappresentata dall'insieme vuoto, poiché all'inizio dell'esecuzione del programma non si assume alcuna informazione sulle variabili. In particolare, non viene considerata costante nessuna variabile senza che ciò sia esplicitamente dimostrato dal codice. Questa scelta evita di derivare conclusioni errate e rende l'analisi conservativa e corretta. Anche gli Initial Interior Points vengono inizializzati con l'insieme vuoto, perché inizialmente non si possiedono informazioni sui valori delle variabili nei vari punti del programma. Partire da insiemi vuoti consente di introdurre solamente informazioni realmente dedotte durante l'analisi del codice, garantendo così una propagazione delle costanti precisa e affidabile.

CFG e Tabella delle iterazioni



	Iterazione 1		Iterazione 2	
	In[b]	Out[b]	In[b]	Out[b]
Entry	{}	{}	{}	{}
k=2	{}	{<k,2>}	{}	{<k,2>}
if	{<k,2>}	{<k,2>}	{<k,2>}	{<k,2>}
a=k+2	{<k,2>}	{<k,2>, <a,4>}	{<k,2>}	{<k,2>, <a,4>}
x=5	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,5>}	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,5>}
a=k*2	{<k,2>}	{<k,2>, <a,4>}	{<k,2>}	{<k,2>, <a,4>}
x=8	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,8>}	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,8>}
k=a	{<k,2>, <a,4>}	{<k,4>, <a,4>}	{<k,2>, <a,4>}	{<k,4>, <a,4>}

while	{<k,4>, <a,4>}	{<k,4>, <a,4>}	{<a,4>}	{<a,4>}
b=2	{<k,4>, <a,4>}	{<k,4>, <a,4>, <b,2>}	{<a,4>}	{<a,4>, <b,2>}
x=a+k	{<k,4>, <a,4>, <b,2>}	{<k,4>, <a,4>, <b,2>, <x,8>}	{<a,4>, <b,2>}	{<a,4>, <b,2>}
y=a*b	{<k,4>, <a,4>, <b,2>, <x,8>}	{<k,4>, <a,4>, <b,2>, <x,8>, <y,8>}	{<a,4>, <b,2>}	{<a,4>, <b,2>, <y,8>}
k++	{<k,4>, <a,4>, <b,2>, <x,8>, <y,8>}	{<a,4>, <b,2>, <x,8>, <y,8>}	{<k,4>, <a,4>, <b,2>, <x,8>, <y,8>}	{<a,4>, <b,2>, <x,8>, <y,8>}
print(a+x)	{<k,4>, <a,4>}	{<k,4>, <a,4>}	{<a,4>}	{<a,4>}
exit	{<k,4>, <a,4>}	{<k,4>, <a,4>}	{<a,4>}	{<a,4>}

*nota: le iterazioni in cui il sistema converge non sono state scritte nella tabella